Document 2 — Full Technical Implementation Plan

OpenHealth

Full Implementation Plan — Architecture, Database, UI, APIs, AI, Security & Hackathon Execution

Document Purpose

This document is the **technical master specification** for OpenHealth.

The Product / Functional Blueprint defined what the platform should do. This document defines **how the complete system should be built**, what data must be stored, how the frontend and backend communicate, how AI fits into the architecture, how healthcare workflows are represented, and how the team should implement the project within a hackathon environment.

The architecture is designed to remain scalable without over-engineering the initial implementation.

---

1. Product Engineering Principle

OpenHealth should be built around one central healthcare decision loop:

```
DISCOVER
   ↓
UNDERSTAND
   ↓
COMPARE
   ↓
VERIFY
   ↓
CHECK AVAILABILITY
   ↓
ESTIMATE COST
   ↓
CHECK COVERAGE
   ↓
RESERVE / DECIDE
   ↓
TREATMENT
   ↓
ANALYZE BILL
   ↓
LEARN
```

The architecture should support this complete loop while allowing individual modules to operate independently.

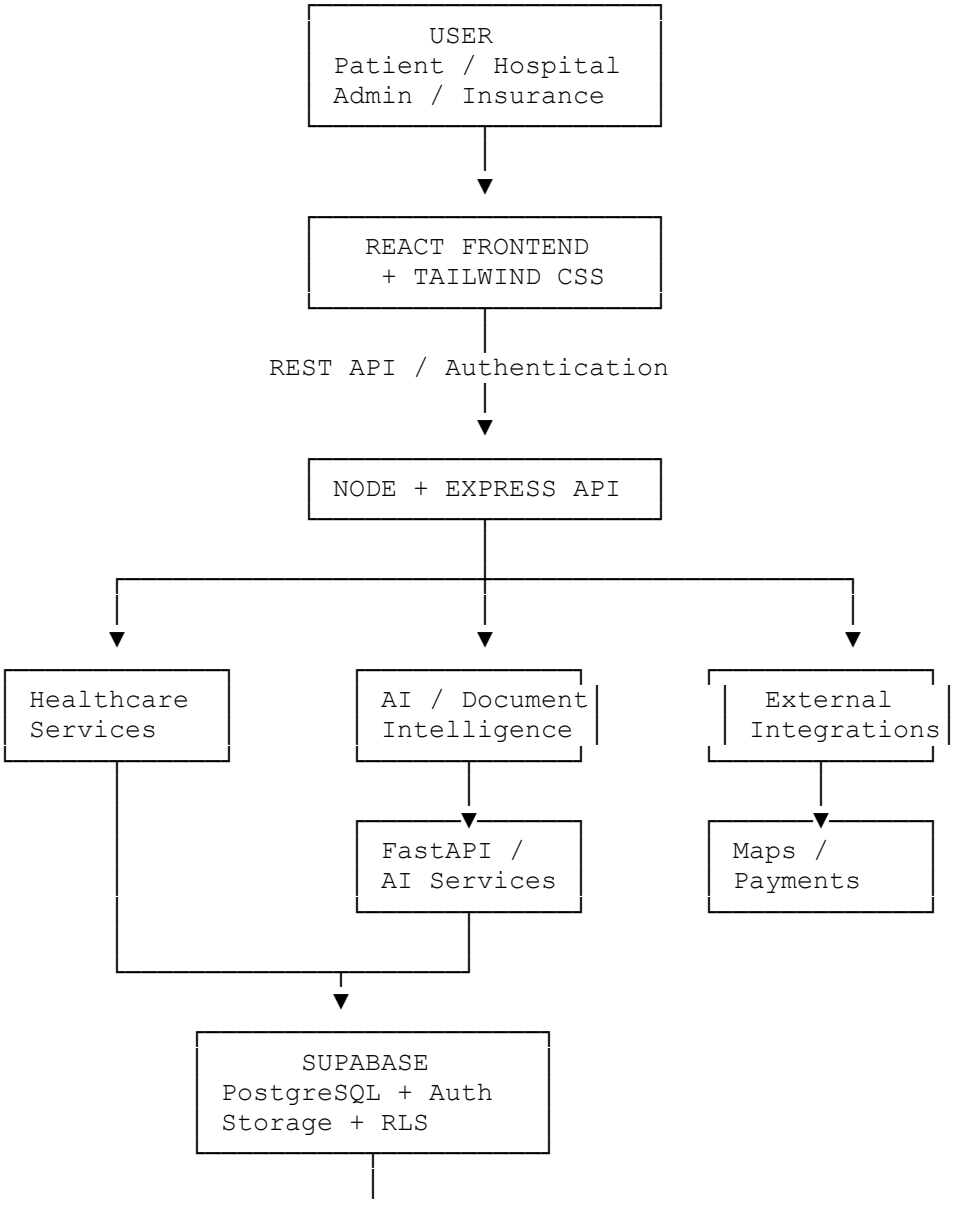
The most important engineering principle is:

Do not build OpenHealth as a collection of unrelated CRUD screens. Build it as an interconnected healthcare decision platform.

---

2. System Architecture

High-Level Architecture



3. Recommended Technology Stack

Frontend

React

Use React for the complete web application.

Tailwind CSS

Use Tailwind CSS for:

- layout
- responsive design
- spacing
- colors
- typography
- states
- responsive breakpoints
- healthcare dashboards

Supporting frontend libraries

Recommended:

- React Router
- Axios or Fetch API
- Recharts
- Lucide React
- React Hook Form
- Zod or lightweight form validation
- TanStack Query where useful

Important decision

No TypeScript.

The frontend should use standard:

```
.jsx
.js
```

files.

Do not introduce `.tsx`, `.ts`, or TypeScript configuration.

---

4. Backend Stack

Node.js + Express

The primary application API will use:

- Node.js
- Express.js
- REST APIs
- JWT authentication
- middleware-based authorization

The backend will handle:

- user operations
- hospitals
- departments
- doctors
- treatments
- packages
- beds
- reservations
- searches
- cost records
- insurance/scheme workflows
- transparency calculations
- bill-analysis orchestration
- report-analysis orchestration

---

5. AI / Document Processing Layer

AI-intensive operations should not clutter the main Express controllers.

Use a dedicated Python service where required.

Recommended AI stack

```
Python
FastAPI
OCR engine
```

```
LLM API
ML libraries where needed
```

The project presentation specifically identifies Python, FastAPI, Tesseract OCR, LLM APIs, and TensorFlow/PyTorch as the AI/document-processing stack.

AI service responsibilities

- medical report extraction
- medical report summarization
- bill OCR
- bill line-item extraction
- bill categorization
- anomaly identification
- healthcare matching assistance
- cost prediction support
- natural-language explanations

---

6. Database

Supabase PostgreSQL

Supabase PostgreSQL should be the primary application database.

Use:

- PostgreSQL
- Supabase Auth
- Row Level Security
- Supabase Storage
- PostgreSQL relationships
- indexes
- constraints
- database functions where useful

The project concept explicitly proposes PostgreSQL through Supabase and PostGIS-style geospatial capability.

---

7. External Integrations

OpenHealth may connect to:

Maps

For:

- distance calculation
- nearby hospitals
- emergency discovery
- route information

Payment Gateway

For:

- booking deposits
- reservation holds
- package deposits

The submitted concept explicitly names Google Maps API and Razorpay/UPI as intended integrations.

Authentication

Supabase Auth + JWT.

AI API

LLM API for structured healthcare explanations and recommendations.

---

8. Application Layers

The entire application should follow this logical structure:

```
Presentation
     ↓
API
     ↓
Controller
     ↓
Service
     ↓
Business Logic
     ↓
Database / External Service
```

For AI operations:

```
Controller
    ↓
```

```
AI Service
    ↓
FastAPI
    ↓
OCR / LLM / ML
    ↓
Structured Result
    ↓
Node Service
    ↓
Database
```

---

9. Frontend Architecture

Frontend Responsibilities

The React application owns:

- page rendering
- navigation
- forms
- user interactions
- validation feedback
- charts
- search interfaces
- hospital comparison
- booking interfaces
- AI result presentation
- loading states
- error states

The frontend should **not** contain sensitive business calculations that belong on the server.

---

10. Frontend Folder Structure

```
openhealth/
│
├── client/
│   │
│   ├── public/
│   │   ├── logo.svg
│   │   ├── favicon.svg
│   │   └── images/
│   │
│   ├── src/
│   │
│   │   ├── assets/
```

```
│   │   │   ├── images/
│   │   │   ├── icons/
│   │   │   └── illustrations/
│   │   │
│   │   ├── components/
│   │   │   │
│   │   │   ├── common/
│   │   │   ├── layout/
│   │   │   ├── dashboard/
│   │   │   ├── search/
│   │   │   ├── hospitals/
│   │   │   ├── doctors/
│   │   │   ├── beds/
│   │   │   ├── bookings/
│   │   │   ├── documents/
│   │   │   ├── bills/
│   │   │   ├── insurance/
│   │   │   ├── packages/
│   │   │   ├── transparency/
│   │   │   ├── emergency/
│   │   │   └── admin/
│   │   │
│   │   ├── pages/
│   │   │   ├── public/
│   │   │   ├── auth/
│   │   │   ├── patient/
│   │   │   ├── hospital/
│   │   │   ├── insurance/
│   │   │   └── admin/
│   │   │
│   │   ├── services/
│   │   ├── hooks/
│   │   ├── context/
│   │   ├── routes/
│   │   ├── lib/
│   │   ├── constants/
│   │   ├── utils/
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── index.css
│   │
│   ├── .env
│   ├── .env.example
│   ├── package.json
│   └── vite.config.js
```

---

11. Frontend Component Organization

Common

```
Button.jsx
Input.jsx
Select.jsx
Modal.jsx
Dropdown.jsx
```

Loader.jsx
ErrorState.jsx
EmptyState.jsx
Toast.jsx
Badge.jsx
Avatar.jsx
FileUploader.jsx

Layout

AppLayout.jsx
Sidebar.jsx
Navbar.jsx
MobileNavbar.jsx
Footer.jsx
Breadcrumb.jsx
ProtectedRoute.jsx

Search

SearchBar.jsx
SearchFilters.jsx
SearchResults.jsx
SearchResultCard.jsx
SearchTypeSelector.jsx
LocationSelector.jsx

Hospital

HospitalCard.jsx
HospitalHeader.jsx
HospitalProfile.jsx
HospitalServices.jsx
HospitalFacilities.jsx
HospitalDoctors.jsx
HospitalPackages.jsx
HospitalAvailability.jsx
HospitalTransparency.jsx
HospitalComparison.jsx

Bed

BedAvailabilityCard.jsx
BedStatusBadge.jsx
BedSearch.jsx
BedTypeSelector.jsx
AvailabilityTimeline.jsx
ReservationCard.jsx

Documents

DocumentUploader.jsx
DocumentPreview.jsx

```
ReportAnalysis.jsx
ExtractedInformation.jsx
AIExplanation.jsx
```

Bills

```
BillUploader.jsx
BillSummary.jsx
BillLineItems.jsx
BillCategoryChart.jsx
BillAnomalyCard.jsx
BillShockIndex.jsx
EstimateVsActual.jsx
```

---

12. Frontend Pages

Public

```
Landing.jsx
Search.jsx
HospitalDiscovery.jsx
HospitalPublicProfile.jsx
HowItWorks.jsx
About.jsx
```

Authentication

```
Login.jsx
Signup.jsx
ForgotPassword.jsx
RoleSelection.jsx
Onboarding.jsx
```

Patient

```
Dashboard.jsx
MyProfile.jsx
SmartCare.jsx
SearchResults.jsx
HospitalDetails.jsx
CompareHospitals.jsx
Emergency.jsx
BedSearch.jsx
BedReservation.jsx
TreatmentDetails.jsx
CostPrediction.jsx
InsuranceChecker.jsx
SchemeChecker.jsx
Documents.jsx
ReportAnalysis.jsx
Bills.jsx
```

```
BillAnalysis.jsx
Bookings.jsx
SavedHospitals.jsx
```

Hospital

```
HospitalDashboard.jsx
HospitalProfile.jsx
ManageBeds.jsx
ManageDoctors.jsx
ManageDepartments.jsx
ManagePackages.jsx
Bookings.jsx
HospitalAnalytics.jsx
TransparencyDashboard.jsx
```

Admin

```
AdminDashboard.jsx
ManageUsers.jsx
ManageHospitals.jsx
VerifyHospitals.jsx
VerifyDoctors.jsx
ManageSchemes.jsx
ManageInsurance.jsx
AuditLogs.jsx
PlatformAnalytics.jsx
```

---

13. Frontend Services

Every API domain gets its own service.

```
api.js
authService.js
userService.js
hospitalService.js
doctorService.js
departmentService.js
treatmentService.js
packageService.js
bedService.js
bookingService.js
searchService.js
documentService.js
reportService.js
billService.js
costService.js
insuranceService.js
schemeService.js
transparencyService.js
emergencyService.js
adminService.js
```

Example:

```
hospitalService.js

getHospitals()
getHospital()
searchHospitals()
compareHospitals()
getHospitalDoctors()
getHospitalPackages()
getHospitalTransparency()
```

Do not put raw API calls directly inside visual components.

---

14. State Management

For the hackathon, avoid unnecessary complexity.

Use:

React Context

For global concerns such as:

```
AuthContext
UserContext
PatientContext
HospitalContext
```

Local State

For:

- forms
- filters
- temporary search state
- comparison selection
- modal states

Server State

Use TanStack Query where useful for:

- hospital queries
- search results
- bed availability
- bookings

- analytics

Do not introduce Redux unless the application genuinely requires it.

---

15. Backend Folder Structure

```
server/
│
├── src/
│
│   ├── config/
│   │   ├── db.js
│   │   ├── env.js
│   │   ├── ai.js
│   │   └── storage.js
│   │
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── userController.js
│   │   ├── hospitalController.js
│   │   ├── doctorController.js
│   │   ├── departmentController.js
│   │   ├── treatmentController.js
│   │   ├── packageController.js
│   │   ├── bedController.js
│   │   ├── bookingController.js
│   │   ├── searchController.js
│   │   ├── documentController.js
│   │   ├── reportController.js
│   │   ├── billController.js
│   │   ├── costController.js
│   │   ├── insuranceController.js
│   │   ├── schemeController.js
│   │   ├── transparencyController.js
│   │   ├── emergencyController.js
│   │   └── adminController.js
│   │
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── userRoutes.js
│   │   ├── hospitalRoutes.js
│   │   ├── doctorRoutes.js
│   │   ├── departmentRoutes.js
│   │   ├── treatmentRoutes.js
│   │   ├── packageRoutes.js
│   │   ├── bedRoutes.js
│   │   ├── bookingRoutes.js
│   │   ├── searchRoutes.js
│   │   ├── documentRoutes.js
│   │   ├── reportRoutes.js
│   │   ├── billRoutes.js
│   │   ├── costRoutes.js
│   │   ├── insuranceRoutes.js
│   │   ├── schemeRoutes.js
```

```
│   │   ├── transparencyRoutes.js
│   │   └── adminRoutes.js
│   │
│   ├── services/
│   │   │
│   │   ├── search/
│   │   │   ├── searchService.js
│   │   │   ├── rankingService.js
│   │   │   └── matchingService.js
│   │   │
│   │   ├── hospital/
│   │   │   ├── hospitalService.js
│   │   │   └── verificationService.js
│   │   │
│   │   ├── beds/
│   │   │   ├── bedService.js
│   │   │   └── availabilityService.js
│   │   │
│   │   ├── bookings/
│   │   │   ├── bookingService.js
│   │   │   └── reservationService.js
│   │   │
│   │   ├── ai/
│   │   │   ├── aiClient.js
│   │   │   ├── promptBuilder.js
│   │   │   └── recommendationService.js
│   │   │
│   │   ├── documents/
│   │   │   ├── documentService.js
│   │   │   └── piiMaskingService.js
│   │   │
│   │   ├── bills/
│   │   │   ├── billService.js
│   │   │   ├── billAnalysisService.js
│   │   │   └── billShockService.js
│   │   │
│   │   ├── reports/
│   │   │   ├── reportService.js
│   │   │   └── reportAnalysisService.js
│   │   │
│   │   ├── costs/
│   │   │   ├── costPredictionService.js
│   │   │   └── pricingService.js
│   │   │
│   │   ├── insurance/
│   │   │   └── insuranceService.js
│   │   │
│   │   ├── schemes/
│   │   │   └── schemeService.js
│   │   │
│   │   └── transparency/
│   │       ├── transparencyService.js
│   │       └── scoreService.js
│   │
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   ├── roleMiddleware.js
│   │   ├── errorMiddleware.js
│   │   ├── validationMiddleware.js
```

```
│   │   ├── uploadMiddleware.js
│   │   └── rateLimitMiddleware.js
│   │
│   ├── validators/
│   │   ├── authValidator.js
│   │   ├── hospitalValidator.js
│   │   ├── doctorValidator.js
│   │   ├── bedValidator.js
│   │   ├── bookingValidator.js
│   │   └── documentValidator.js
│   │
│   ├── constants/
│   │   ├── userRoles.js
│   │   ├── bookingStatuses.js
│   │   ├── bedStatuses.js
│   │   ├── documentTypes.js
│   │   └── transparencyWeights.js
│   │
│   ├── utils/
│   │   ├── response.js
│   │   ├── logger.js
│   │   ├── pagination.js
│   │   └── errors.js
│   │
│   ├── app.js
│   └── server.js
│
├── package.json
└── .env.example
```

---

16. Database Architecture

The database should be relational because OpenHealth contains highly connected healthcare entities.

The central relationship is:

```
USER
 ↓
PATIENT / HOSPITAL / INSURANCE / ADMIN
 ↓
HOSPITAL
 ↓
DEPARTMENTS
 ↓
DOCTORS
 ↓
TREATMENTS
 ↓
PACKAGES
 ↓
BEDS
 ↓
BOOKINGS

PATIENT
```

```
 ↓
DOCUMENTS
 ↓
REPORT ANALYSIS
 ↓
BILL ANALYSIS
 ↓
COST / COVERAGE / TRANSPARENCY
```

---

17. Core Database Tables

17.1 profiles

Stores application users.

Field	Type	Purpose
id	UUID	Primary key
full_name	TEXT	User name
email	TEXT	User email
phone	TEXT	Contact
role	TEXT	User role
avatar_url	TEXT	Profile image
created_at	TIMESTAMP	Creation
updated_at	TIMESTAMP	Last update

Roles

```
patient
hospital_staff
hospital_admin
insurance_user
platform_admin
```

---

18. patient_profiles

Patient-specific information.

Field	Type
id	UUID
user_id	UUID
date_of_birth	DATE
gender	TEXT
city	TEXT

Field	Type
emergency_contact	TEXT
preferred_language	TEXT
created_at	TIMESTAMP

Sensitive patient information should be minimized.

---

19. hospitals

Represents healthcare institutions.

Field	Type	Purpose
id	UUID	Primary key
name	TEXT	Hospital name
type	TEXT	Hospital type
description	TEXT	Description
address	TEXT	Address
city	TEXT	City
state	TEXT	State
latitude	NUMERIC	Location
longitude	NUMERIC	Location
phone	TEXT	Contact
website	TEXT	Website
verification_status	TEXT	Verified state
transparency_score	NUMERIC	Current score
created_at	TIMESTAMP	Creation
updated_at	TIMESTAMP	Update

---

20. hospital_users

Connects users to hospitals.

Field	Type
id	UUID
hospital_id	UUID
user_id	UUID
role	TEXT

Field	Type
created_at	TIMESTAMP

This permits multiple staff members within a hospital.

---

21. departments

Represents hospital departments.

Examples:

```
Cardiology
Neurology
Orthopedics
Oncology
General Medicine
Emergency Medicine
```

Field	Type
id	UUID
hospital_id	UUID
name	TEXT
description	TEXT
emergency_available	BOOLEAN
created_at	TIMESTAMP

---

22. doctors

Represents healthcare professionals.

Field	Type
id	UUID
hospital_id	UUID
department_id	UUID
name	TEXT
specialization	TEXT
qualification	TEXT
experience_years	INTEGER
registration_number	TEXT
consultation_fee	NUMERIC
verification_status	TEXT

Field	Type
created_at	TIMESTAMP

---

23. treatments

Represents procedures and services.

Field	Type
id	UUID
name	TEXT
category	TEXT
department_id	UUID
description	TEXT
created_at	TIMESTAMP

---

24. hospital_treatments

Maps treatments to hospitals.

Field	Type
id	UUID
hospital_id	UUID
treatment_id	UUID
available	BOOLEAN
estimated_min_cost	NUMERIC
estimated_max_cost	NUMERIC
created_at	TIMESTAMP

---

25. treatment_packages

Represents fixed or semi-fixed hospital packages.

Field	Type
id	UUID
hospital_id	UUID
treatment_id	UUID
name	TEXT

Field	Type
price	NUMERIC
duration_days	INTEGER
room_category	TEXT
included_services	JSONB
excluded_services	JSONB
package_lock_available	BOOLEAN
emi_available	BOOLEAN
active	BOOLEAN
created_at	TIMESTAMP
updated_at	TIMESTAMP

---

26. bed_types

Defines available resource categories.

Field	Type
id	UUID
name	TEXT
description	TEXT

Examples:

```
General
ICU
NICU
PICU
HDU
Isolation
```

---

27. hospital_beds

Represents hospital bed inventory.

Field	Type
id	UUID
hospital_id	UUID
bed_type_id	UUID
total_beds	INTEGER
occupied_beds	INTEGER

Field	Type
reserved_beds	INTEGER
available_beds	INTEGER
last_updated_at	TIMESTAMP

Available beds should preferably be derived or validated rather than blindly trusted.

---

28. bed_reservations

Represents resource reservations.

Field	Type
id	UUID
patient_id	UUID
hospital_id	UUID
bed_type_id	UUID
status	TEXT
deposit_amount	NUMERIC
payment_status	TEXT
reserved_at	TIMESTAMP
expires_at	TIMESTAMP
confirmed_at	TIMESTAMP

Status

```
pending
held
confirmed
expired
cancelled
completed
```

---

29. bookings

General healthcare booking entity.

Field	Type
id	UUID
patient_id	UUID
hospital_id	UUID

Field	Type
treatment_id	UUID
package_id	UUID
booking_type	TEXT
status	TEXT
amount	NUMERIC
payment_status	TEXT
created_at	TIMESTAMP

---

30. medical_documents

Stores metadata for uploaded documents.

Field	Type
id	UUID
patient_id	UUID
document_type	TEXT
file_url	TEXT
original_filename	TEXT
mime_type	TEXT
masked	BOOLEAN
processing_status	TEXT
uploaded_at	TIMESTAMP

Document types

```
medical_report
prescription
discharge_summary
medical_bill
other
```

The actual document should be stored in secure object storage rather than directly inside PostgreSQL.

---

31. report_analyses

Stores extracted/processed report information.

Field	Type
id	UUID
document_id	UUID
summary	TEXT
extracted_data	JSONB
detected_conditions	JSONB
detected_specialties	JSONB
ai_explanation	TEXT
model_version	TEXT
created_at	TIMESTAMP

---

32. bills

Represents uploaded medical bills.

Field	Type
id	UUID
patient_id	UUID
hospital_id	UUID
document_id	UUID
bill_number	TEXT
bill_date	DATE
estimated_amount	NUMERIC
final_amount	NUMERIC
insurance_amount	NUMERIC
patient_payable	NUMERIC
created_at	TIMESTAMP

---

33. bill_line_items

Stores extracted individual charges.

Field	Type
id	UUID
bill_id	UUID
category	TEXT
description	TEXT
quantity	NUMERIC

Field	Type
unit_price	NUMERIC
amount	NUMERIC
extracted_confidence	NUMERIC
anomaly_flag	BOOLEAN

---

34. bill_analyses

Stores AI bill analysis.

Field	Type
id	UUID
bill_id	UUID
total_detected	NUMERIC
categorized_total	NUMERIC
suspicious_amount	NUMERIC
anomaly_count	INTEGER
analysis_summary	TEXT
model_version	TEXT
created_at	TIMESTAMP

---

35. bill_shock_records

Stores estimate-versus-final analysis.

Field	Type
id	UUID
hospital_id	UUID
bill_id	UUID
estimated_amount	NUMERIC
final_amount	NUMERIC
variance_amount	NUMERIC
variance_percent	NUMERIC
shock_level	TEXT
created_at	TIMESTAMP

Shock level

```
low
```

```
moderate
high
```

---

36. cost_predictions

Stores treatment-cost predictions.

Field	Type
id	UUID
patient_id	UUID
hospital_id	UUID
treatment_id	UUID
min_cost	NUMERIC
max_cost	NUMERIC
confidence	NUMERIC
inputs	JSONB
model_version	TEXT
created_at	TIMESTAMP

---

37. insurance_providers

Field	Type
id	UUID
name	TEXT
description	TEXT
contact	TEXT
active	BOOLEAN

---

38. insurance_policies

Field	Type
id	UUID
patient_id	UUID
provider_id	UUID
policy_number	TEXT
plan_name	TEXT
status	TEXT

Field	Type
start_date	DATE
end_date	DATE

Sensitive policy information should receive additional protection.

---

39. government_schemes

Field	Type
id	UUID
name	TEXT
description	TEXT
eligibility_rules	JSONB
covered_treatments	JSONB
active	BOOLEAN

---

40. eligibility_checks

Represents an eligibility evaluation.

Field	Type
id	UUID
patient_id	UUID
hospital_id	UUID
treatment_id	UUID
insurance_provider_id	UUID
scheme_id	UUID
eligible	BOOLEAN
coverage_amount	NUMERIC
copay_amount	NUMERIC
explanation	TEXT
created_at	TIMESTAMP

---

41. transparency_scores

Stores hospital transparency history.

Field	Type
id	UUID
hospital_id	UUID
price_clarity_score	NUMERIC
package_clarity_score	NUMERIC
information_score	NUMERIC
data_freshness_score	NUMERIC
billing_consistency_score	NUMERIC
verification_score	NUMERIC
overall_score	NUMERIC
scoring_version	TEXT
calculated_at	TIMESTAMP

---

42. hospital_metrics

Stores aggregated platform metrics.

Field	Type
id	UUID
hospital_id	UUID
period_start	DATE
period_end	DATE
searches	INTEGER
profile_views	INTEGER
booking_count	INTEGER
package_views	INTEGER
bed_queries	INTEGER
created_at	TIMESTAMP

---

43. notifications

Field	Type
id	UUID
user_id	UUID
type	TEXT
title	TEXT
message	TEXT
read	BOOLEAN

Field	Type
created_at	TIMESTAMP

---

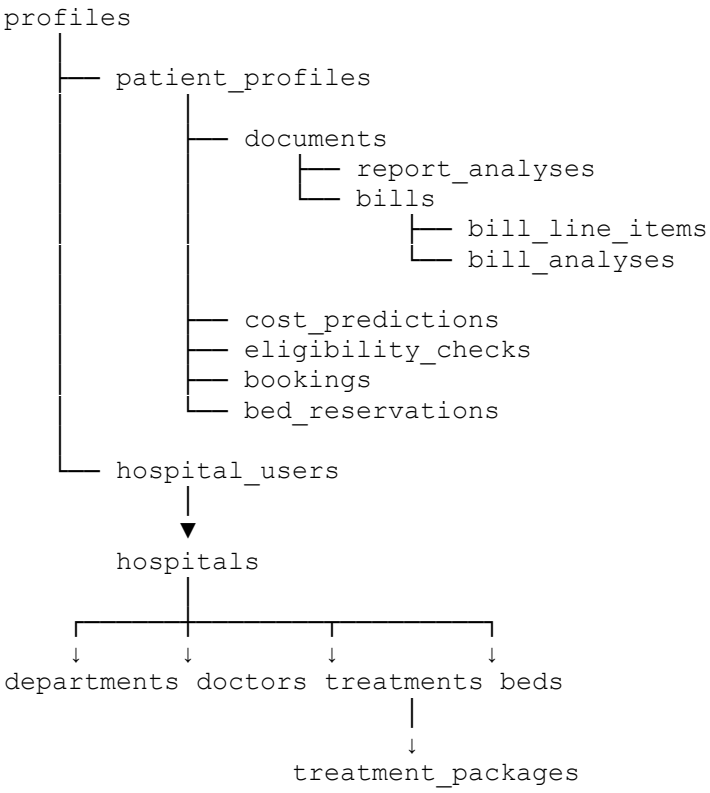
44. audit_logs

This is especially important for healthcare-sensitive operations.

Field	Type
id	UUID
user_id	UUID
action	TEXT
entity_type	TEXT
entity_id	UUID
metadata	JSONB
ip_address	TEXT
created_at	TIMESTAMP

---

45. Core Database Relationship



---

46. API Architecture

Base URL:

```
/api
```

---

47. Authentication APIs

```
POST   /api/auth/register
POST   /api/auth/login
POST   /api/auth/logout
GET    /api/auth/me
POST   /api/auth/refresh
POST   /api/auth/forgot-password
```

Supabase Auth can handle the actual authentication mechanism while Express verifies authenticated sessions/tokens for protected operations.

---

48. User APIs

```
GET    /api/users/me
PATCH  /api/users/me
GET    /api/users/me/profile
PATCH  /api/users/me/profile
```

---

49. Hospital APIs

```
GET    /api/hospitals
GET    /api/hospitals/:id
POST   /api/hospitals
PATCH  /api/hospitals/:id
DELETE /api/hospitals/:id

GET    /api/hospitals/:id/departments
GET    /api/hospitals/:id/doctors
GET    /api/hospitals/:id/treatments
GET    /api/hospitals/:id/packages
GET    /api/hospitals/:id/transparency
GET    /api/hospitals/:id/availability
```

---

50. Search APIs

```
GET /api/search
```

```
GET /api/search/hospitals
GET /api/search/doctors
GET /api/search/treatments
GET /api/search/departments
POST /api/search/smart-match
```

Example:

```
GET /api/search/hospitals?
query=knee%20pain&
lat=22.7&
lng=75.8&
radius=20
```

---

51. Bed APIs

```
GET   /api/beds/search
GET   /api/hospitals/:id/beds
PATCH /api/hospitals/:id/beds
POST  /api/beds/reservations
GET   /api/beds/reservations/:id
PATCH /api/beds/reservations/:id
```

---

52. Booking APIs

```
GET   /api/bookings
POST  /api/bookings
GET   /api/bookings/:id
PATCH /api/bookings/:id
POST  /api/bookings/:id/cancel
```

---

53. Package APIs

```
GET   /api/packages
GET   /api/packages/:id
POST  /api/hospitals/:id/packages
PATCH /api/packages/:id
DELETE /api/packages/:id
```

---

54. Medical Document APIs

```
POST /api/documents
GET  /api/documents
GET  /api/documents/:id
DELETE /api/documents/:id
POST /api/documents/:id/analyze
```

---

55. Report Analysis APIs

```
POST /api/reports/analyze
GET  /api/reports/:id
```

The response should be structured rather than returning an unprocessed AI paragraph.

Example conceptual response:

```
{
  summary,
  detectedConditions,
  specialties,
  extractedValues,
  importantTerms,
  confidence
}
```

---

56. Bill APIs

```
POST   /api/bills
GET    /api/bills
GET    /api/bills/:id
POST   /api/bills/:id/analyze
GET    /api/bills/:id/line-items
GET    /api/bills/:id/shock-index
```

---

57. Cost Prediction APIs

```
POST /api/cost/predict
GET  /api/cost/predictions
GET  /api/cost/predictions/:id
```

---

58. Insurance APIs

```
GET  /api/insurance/providers
POST /api/insurance/check
GET  /api/insurance/checks
GET  /api/insurance/checks/:id
```

---

59. Government Scheme APIs

```
GET  /api/schemes
GET  /api/schemes/:id
POST /api/schemes/check-eligibility
GET  /api/schemes/eligible
```

---

60. Transparency APIs

```
GET /api/transparency/hospitals
GET /api/transparency/hospitals/:id
GET /api/transparency/hospitals/:id/history
GET /api/transparency/leaderboard
```

---

61. Emergency APIs

```
GET  /api/emergency/search
POST /api/emergency/reserve
GET  /api/emergency/nearby
```

Emergency search should optimize primarily for:

```
availability
+
medical suitability
+
distance
+
verification
```

---

62. Admin APIs

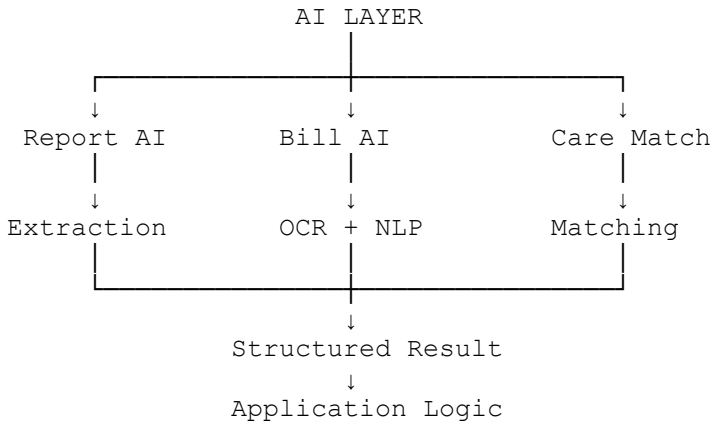
```
GET   /api/admin/dashboard
GET   /api/admin/users
GET   /api/admin/hospitals
PATCH /api/admin/hospitals/:id/verify
GET   /api/admin/doctors
PATCH /api/admin/doctors/:id/verify
GET   /api/admin/audit-logs
GET   /api/admin/analytics
```

---

63. AI Architecture

AI should not be implemented as one giant chatbot.

OpenHealth should use specialized AI capabilities.



---

64. AI Service Structure

```
ai/
├── aiClient.js
├── promptBuilder.js
├── reportAnalyzer.js
├── billAnalyzer.js
├── careMatcher.js
├── costPredictor.js
└── explanationService.js
```

---

65. Medical Report AI Flow

```
React
 ↓
Upload Document
 ↓
Node API
 ↓
Storage
 ↓
FastAPI
 ↓
OCR / Extraction
 ↓
Structured Medical Information
 ↓
LLM Interpretation
 ↓
Node API
 ↓
Database
 ↓
React
```

---

66. Bill AI Flow

```
Bill Upload
     ↓
PII Masking
     ↓
OCR
     ↓
Text Extraction
     ↓
Line Item Extraction
     ↓
Category Classification
     ↓
Anomaly Detection
     ↓
Estimate vs Final
     ↓
Bill Shock Index
     ↓
Patient-Friendly Explanation
```

---

67. AI Boundaries

AI should **assist**, not silently invent healthcare facts.

The platform should distinguish between:

Extracted fact

Information actually present in the document.

Calculated result

Mathematically derived by the application.

AI interpretation

Generated explanation or categorization.

Prediction

A model-based estimate.

Each should be identifiable in the product.

---

68. Cost Prediction Architecture

The cost prediction system should not simply ask an LLM:

“What will this surgery cost?”

Instead:

```
Treatment
+
Hospital
+
Package Data
+
Historical Pricing
+
Patient Context
+
Room Category
+
Expected Duration
        ↓
Cost Prediction Service
        ↓
Estimated Range
```

The AI layer can help interpret the result, but the system should preserve the underlying structured inputs.

---

69. Smart Healthcare Matching

Smart matching should combine deterministic and AI-assisted logic.

```
Patient Input
      ↓
Symptom / Treatment Extraction
      ↓
Department Mapping
      ↓
Hospital Capability Filter
      ↓
Location Filter
      ↓
Availability Filter
      ↓
Insurance / Scheme Filter
      ↓
Transparency Ranking
      ↓
Recommended Hospitals
```

---

70. Hospital Ranking

The ranking engine can use:

```
Medical relevance
+
Distance
+
Availability
+
Doctor relevance
+
Cost compatibility
+
Insurance compatibility
+
Scheme support
+
Transparency
+
Verification
```

The final result should be explainable.

Example:

```
Why this hospital?

✓ Relevant Orthopedic department
✓ Treatment available
✓ ICU available
✓ 4.8 km away
✓ Insurance supported
✓ High transparency score
```

---

71. Transparency Score Engine

The server should calculate transparency.

Possible weighted factors:

```
Price Clarity
Package Clarity
Data Freshness
Information Completeness
Billing Consistency
Verification
```

Example:

```
Price Clarity        20%
Package Clarity      15%
Data Freshness       15%
Information          15%
Billing Consistency  20%
Verification         15%
```

Weights should be stored in configuration rather than scattered throughout frontend components.

---

72. Bill Shock Calculation

The basic deterministic calculation is:

```
variance_amount
=
final_amount - estimated_amount
```

and:

```
variance_percent
=
((final_amount - estimated_amount)
 / estimated_amount) × 100
```

The resulting variance can then be classified into product-defined levels.

The submitted project concept specifically defines the Bill Shock Index around initial estimate versus final bill accuracy.

---

73. Bed Availability Engine

The system should not simply display:

```
ICU = 4
```

without tracking its state.

Conceptually:

```
Total Beds
   -
Occupied
   -
Reserved
   =
Available
```

Each availability record also needs a:

`last_updated_at`

timestamp.

This allows the UI to communicate how fresh the information is.

---

74. Booking Flow

```
Patient
 ↓
Hospital Search
 ↓
Hospital Profile
 ↓
Select Bed / Package
 ↓
Check Availability
 ↓
Create Reservation
 ↓
Payment / Deposit
 ↓
Temporary Hold
 ↓
Hospital Confirmation
 ↓
Booking Confirmed
```

Reservation expiry must be handled server-side.

---

75. Authentication & Authorization

The security model is:

```
React
 ↓
Supabase Auth
 ↓
Authenticated User
 ↓
JWT
 ↓
Express
 ↓
Auth Middleware
 ↓
```

```
Role Middleware
 ↓
Service
 ↓
Database
```

The presentation explicitly proposes Supabase Auth/JWT and role-based separation between patients, hospitals and administrators.

---

76. Role Permissions

Patient

Can:

- search
- compare
- view hospitals
- upload documents
- analyze bills
- check eligibility
- reserve beds
- book packages

Cannot:

- modify hospital data
- modify bed inventory
- access another patient's documents

Hospital Staff

Can:

- update assigned resources
- manage bookings
- update bed status

Cannot:

- access unrelated patient data

Hospital Admin

Can:

- manage hospital information
- manage doctors
- manage departments
- manage packages
- manage beds
- view hospital analytics

Insurance User

Can:

- manage insurance-related workflows
- process eligibility information available to them

Platform Admin

Can:

- verify hospitals
- verify doctors
- moderate data
- manage platform configuration
- view audit information

---

77. Row Level Security

Supabase RLS should enforce data ownership.

Examples:

A patient can read only their documents.

A hospital user can modify only the hospital they belong to.

A platform administrator can access administrative records.

RLS should not be treated as a frontend feature. It must be enforced at the database layer.

---

78. Privacy Architecture

For uploaded healthcare documents:

```
Upload
 ↓
Authentication check
 ↓
File validation
 ↓
Secure storage
 ↓
PII masking
 ↓
Processing
 ↓
Structured analysis
 ↓
Restricted result
```

PII masking is explicitly identified in the submitted project concept.

---

79. Data Classification

The system should conceptually classify information as:

Public

- hospital name
- location
- public departments
- public doctors
- public packages

Restricted

- booking data
- hospital operational data
- insurance information

Highly Sensitive

- medical reports
- bills
- patient identifiers
- personal healthcare information

Different access policies should apply to each class.

---

80. API Request Flow Example — Smart Search

```
User enters:
"Knee replacement near me"
            ↓
SearchBar.jsx
            ↓
searchService.js
            ↓
GET /api/search/smart-match
            ↓
searchController
            ↓
matchingService
            ↓
Treatment / Department Mapping
            ↓
Hospital Filtering
            ↓
Availability Check
            ↓
Transparency Ranking
            ↓
Response
            ↓
SearchResults.jsx
```

---

81. API Request Flow Example — Bill Analysis

```
User uploads bill
        ↓
BillUploader.jsx
        ↓
billService.js
        ↓
POST /api/bills/:id/analyze
        ↓
billController
        ↓
billAnalysisService
        ↓
FastAPI
        ↓
OCR
        ↓
Extraction
        ↓
LLM / Classification
        ↓
Structured Line Items
        ↓
bill_analyses
        ↓
BillAnalysis.jsx
```

82. API Request Flow Example — Emergency Bed Search

```
Emergency.jsx
      ↓
emergencyService.js
      ↓
GET /api/emergency/search
      ↓
emergencyController
      ↓
availabilityService
      ↓
hospital capability filter
      ↓
distance calculation
      ↓
bed availability
      ↓
ranking
      ↓
nearest suitable hospitals
      ↓
Emergency UI
```

---

83. API Request Flow Example — Cost Prediction

```
Patient selects treatment
        ↓
CostPredictionForm
        ↓
costService.js
        ↓
POST /api/cost/predict
        ↓
costController
        ↓
costPredictionService
        ↓
Structured Pricing Data
        ↓
AI / Prediction Layer
        ↓
Estimated Range
        ↓
Database
        ↓
Frontend
```

---

84. API Request Flow Example — Insurance Eligibility

```
Treatment
   +
Hospital
   +
Insurance / Scheme
        ↓
Eligibility Form
        ↓
insuranceService.js
        ↓
POST /api/insurance/check
        ↓
insuranceController
        ↓
eligibilityService
        ↓
Policy / Scheme Rules
        ↓
Coverage Result
        ↓
Patient Contribution
```

---

85. Application Dashboard Data Flow

The dashboard should not make twenty unrelated API requests during initial load.

Prefer grouped data.

```
GET /api/dashboard
```

Response can provide:

```
patient profile
recent searches
saved hospitals
active bookings
recent documents
recent analyses
coverage status
notifications
```

For hospital users:

```
GET /api/hospital-dashboard
```

can provide:

```
bed metrics
active reservations
package metrics
profile completeness
transparency score
recent activity
```

---

86. Error Handling

All API endpoints should return consistent responses.

Example:

```
{
  success: false,
  message: "Hospital not found",
  errorCode: "HOSPITAL_NOT_FOUND"
}
```

Success:

```
{
  success: true,
  data: {...}
}
```

Never leak internal stack traces to the frontend.

---

87. Frontend UX States

Every important screen should have:

Loading

```
Finding hospitals near you...
```

Empty

```
No hospitals matched your current criteria.
```

Processing

```
Analyzing your medical bill...
```

Error

```
We couldn't complete the analysis.
Please try again.
```

Success

```
Bill analysis completed.
```

Sensitive processing

```
Your document is being processed securely.
```

---

88. Responsive Architecture

Desktop is the primary demonstration target.

Desktop

```
Sidebar
+
Navbar
+
Main content
```

Tablet

```
Collapsible navigation
```

Mobile

```
Top navigation
+
Drawer
+
Single-column content
```

Emergency Mode should be particularly optimized for mobile access.

---

89. Search Architecture

Search should support:

```
Free-text
Symptoms
Treatments
Departments
Doctors
```

```
Hospitals
Location
```

Filters:

```
Distance
Bed availability
Price
Insurance
Scheme
Hospital type
Transparency
Verification
```

---

90. Comparison Engine

Patients should be able to select multiple hospitals.

Example:

Hospital A	Hospital B	Hospital C	
Cost	₹2.1L	₹2.3L	₹1.9L
ICU	3 available	1 available	5 available
Doctors	8	5	7
Insurance	✓	✓	×
Package	✓	✓	✓
Transparency	88	82	74
Distance	4 km	7 km	5 km

The data should come from normalized backend entities.

---

91. Reporting

Reporting should initially focus on useful web-based reports.

Patient report

Could include:

- hospital comparison
- cost estimate
- coverage result
- booking information

Bill analysis report

Could include:

- total amount
- extracted categories
- anomalies
- estimate vs final
- Bill Shock Index

Hospital report

Could include:

- profile completeness
- transparency score
- bed demand
- booking activity
- package activity

---

92. Deployment Architecture

Frontend

```
React + Vite
      ↓
Vercel
```

Backend

```
Node + Express
      ↓
Cloud deployment
```

AI Service

```
FastAPI
      ↓
Cloud / Container
```

Database

```
Supabase PostgreSQL
```

Storage

```
Supabase Storage
```

The original presentation proposes Vercel/AWS-style cloud deployment, Docker/Nginx, and Supabase-backed architecture.

---

93. Project Root Structure

```
OpenHealth/
│
├── client/
│
├── server/
│
├── ai-service/
│
├── database/
│   ├── schema.sql
│   ├── seed.sql
│   └── README.md
│
├── docs/
│   ├── architecture.md
│   ├── api.md
│   ├── database.md
│   ├── ai.md
│   └── security.md
│
├── .env.example
├── .gitignore
├── package.json
└── README.md
```

---

94. Database Folder

```
database/
├── schema.sql
├── seed.sql
└── README.md
```

schema.sql

Contains:

- tables
- foreign keys
- constraints

- indexes
- RLS policies

seed.sql

For hackathon demonstration:

- demo patients
- demo hospitals
- departments
- doctors
- treatments
- packages
- bed availability
- insurance providers
- schemes
- historical bills
- transparency scores

This allows the entire demo environment to be recreated quickly.

---

95. Recommended Indexes

Important searchable fields should have indexes.

Examples:

```
hospitals(city)
hospitals(latitude, longitude)
departments(name)
doctors(specialization)
treatments(name)
hospital_beds(hospital_id, bed_type_id)
bed_reservations(status)
bookings(patient_id)
medical_documents(patient_id)
bills(patient_id)
transparency_scores(hospital_id)
```

Search and availability are expected to be high-frequency operations, so these should be considered early.

---

96. Data Consistency Rules

Bed availability

Never allow:

```
available_beds < 0
occupied_beds > total_beds
reserved_beds > total_beds
```

Booking

A cancelled booking must not continue consuming a resource.

Reservation

Expired holds must release resources.

Bill

Final bill amount should not automatically overwrite original estimates.

Hospital verification

Only authorized roles can change verification status.

---

97. Core Engineering Rules

Keep business logic out of React components.

Keep database queries out of route definitions.

Keep AI prompts out of random controllers.

Keep financial calculations deterministic where possible.

Keep patient documents private.

Do not expose internal IDs unnecessarily to public pages.

Never trust client-side role information.

Validate every uploaded file.

Never rely only on frontend authorization.

---

98. Hackathon MVP

The entire vision is large.

The hackathon MVP should focus on one **complete end-to-end healthcare decision journey** rather than implementing every planned feature superficially.

MVP Flow

```
Patient
 ↓
Smart Search
 ↓
Hospital Comparison
 ↓
Live Bed Availability
 ↓
Treatment Cost Estimate
 ↓
Insurance / Scheme Check
 ↓
AI Bill / Report Analysis
 ↓
Transparency Score
 ↓
Reservation
```

This provides a coherent demonstration.

---

99. Priority 1 — Must Work

Patient side

- login
- smart hospital search
- hospital profiles
- comparison
- live bed availability
- emergency mode
- treatment cost estimation
- bill OCR
- report analysis
- insurance/scheme check
- transparency score
- bookings

Hospital side

- hospital login
- hospital profile
- bed management
- package management
- booking management

Backend

- auth
- search
- hospital
- beds
- booking
- documents
- AI
- costs
- transparency

---

100. Priority 2 — Strong Demo Enhancements

- doctor comparison
- richer package comparison
- Bill Shock Index
- patient dashboard
- hospital analytics
- notifications
- saved hospitals
- map visualization
- better AI explanations
- improved emergency ranking

---

101. Priority 3 — Future Expansion

- telemedicine
- e-pharmacy
- claims automation
- advanced predictive risk scoring
- regional-language support
- wearable integrations
- ambulance-bed synchronization
- national healthcare-network expansion

These future capabilities align with the roadmap proposed in the original project presentation.

---

102. Suggested Hackathon Development Phases

Phase 1 — Foundation

Build:

```
Repository
Frontend
Backend
Supabase
Authentication
Base layout
Routing
```

Phase 2 — Healthcare Discovery

Build:

```
Hospital database
Departments
Doctors
Treatments
Search
Filters
Hospital profile
Comparison
```

Phase 3 — Bed Intelligence

Build:

```
Bed tables
Bed dashboard
Availability
Emergency search
Reservation
```

Phase 4 — Financial Intelligence

Build:

```
Packages
```

```
Cost prediction
Insurance
Government schemes
Eligibility
```

Phase 5 — AI Intelligence

Build:

```
Document upload
OCR
Report extraction
Bill extraction
Bill analysis
AI explanation
```

Phase 6 — Transparency

Build:

```
Transparency Score
Bill Shock Index
Hospital transparency page
```

Phase 7 — Polish

Build:

```
Responsive UI
Loading states
Empty states
Error handling
Animations
Charts
Notifications
Demo data
```

---

103. Demo Data Strategy

A hackathon application should never depend entirely on live hospital integrations to demonstrate its core experience.

Seed the database with realistic demo data.

Demo hospitals

Create several different hospital profiles:

```
Large Multi-Specialty Hospital
Specialized Cardiac Hospital
Orthopedic Center
Affordable Community Hospital
```

Demo resource conditions

Use different availability scenarios:

```
Hospital A → ICU Available
Hospital B → ICU Limited
Hospital C → ICU Full
Hospital D → General Beds Available
```

Demo financial scenarios

Include:

```
Low-cost hospital
Premium hospital
Fixed package hospital
High-variance hospital
```

This makes the comparison and transparency functionality visible immediately.

---

104. Core Demo Scenario

The strongest hackathon presentation should follow one patient.

```
Patient enters:
"My mother needs knee replacement."
        ↓
Smart Care Match
        ↓
Orthopedic hospitals
        ↓
Compare 3 hospitals
        ↓
Hospital A
Package: ₹2.1L
Transparency: 88
Insurance: Supported
        ↓
Cost Prediction
₹1.9L – ₹2.3L
        ↓
Coverage Check
₹1.5L covered
        ↓
```

```
Patient contribution
₹50k–₹80k
        ↓
Book / Reserve
        ↓
Later upload bill
        ↓
AI extracts charges
        ↓
Final amount compared
        ↓
Bill Shock Index
        ↓
Transparency insight
```

This demonstrates nearly the entire platform in one coherent story.

---

105. Core Files to Build First

Frontend

```
App.jsx
Dashboard.jsx
Search.jsx
SearchResults.jsx
HospitalDetails.jsx
CompareHospitals.jsx
Emergency.jsx
BedSearch.jsx
BedReservation.jsx
CostPrediction.jsx
InsuranceChecker.jsx
ReportAnalysis.jsx
BillAnalysis.jsx
Bookings.jsx

hospitalService.js
searchService.js
bedService.js
bookingService.js
documentService.js
billService.js
costService.js
insuranceService.js
```

Backend

```
hospitalController.js
searchController.js
bedController.js
bookingController.js
documentController.js
```

```
billController.js
costController.js
insuranceController.js
transparencyController.js

searchService.js
matchingService.js
availabilityService.js
bookingService.js
billAnalysisService.js
costPredictionService.js
transparencyService.js
```

AI

```
reportAnalyzer.py
billAnalyzer.py
ocrService.py
```

Database

```
schema.sql
seed.sql
```

---

106. Source of Truth Architecture

The final implementation should maintain these documents:

```
docs/
│
├── prd.md
├── architecture.md
├── database.md
├── api.md
├── ai.md
├── security.md
├── design.md
├── rules.md
└── phases.md
```

During development, add:

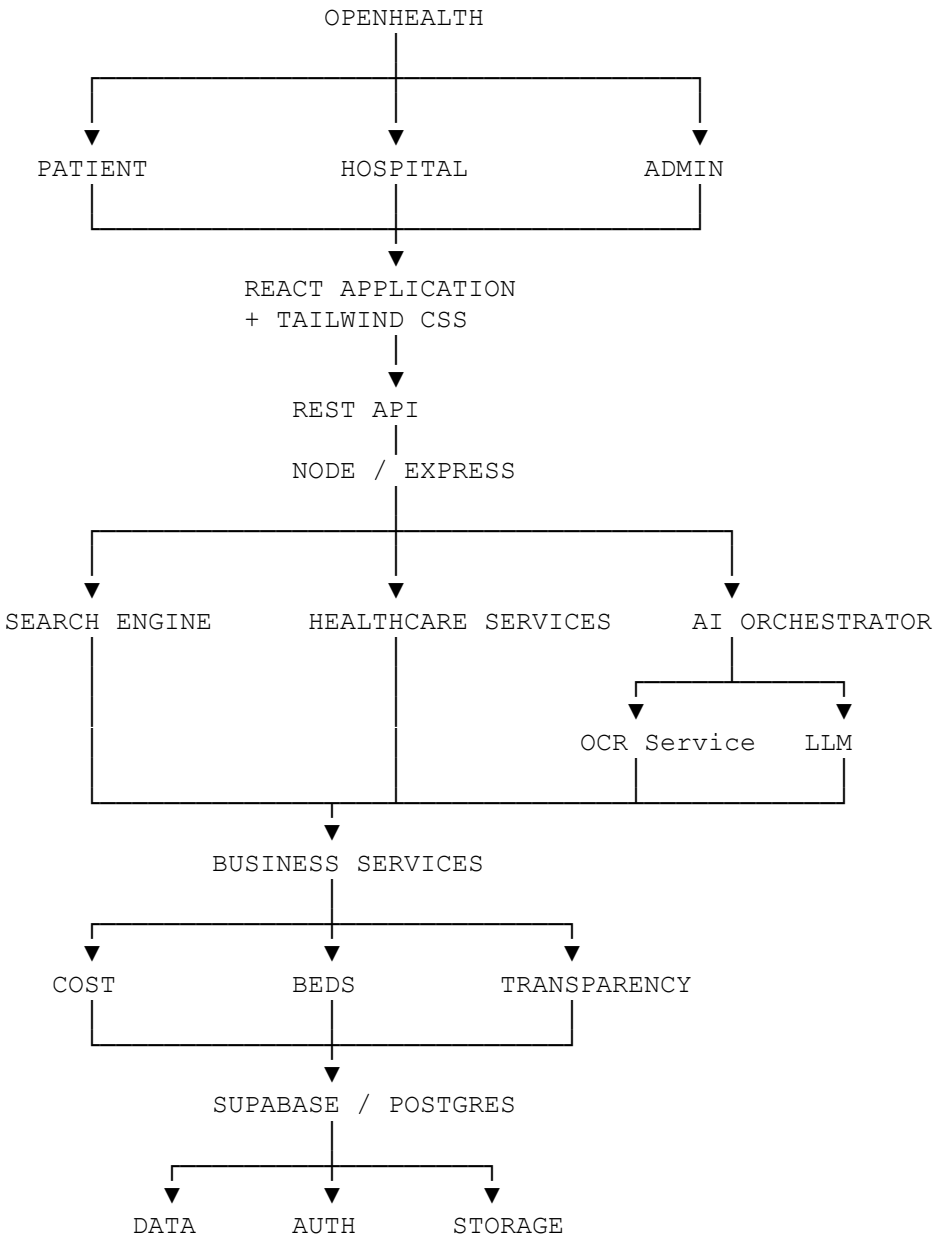
```
memory.md
```

This should track meaningful implementation changes, decisions, completed modules, known issues, and current project state.

The reference architecture follows the same principle of separating product, architecture, coding rules, phases, design, and persistent development state.

---

107. Final Technical Architecture

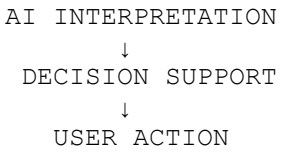


---

108. Final Engineering Principle

The most important architectural rule for OpenHealth is:

```
USER EXPERIENCE
      ↓
HEALTHCARE DATA
      ↓
DETERMINISTIC LOGIC
      ↓
```



AI should enhance the healthcare decision process, while **database-backed facts, explicit business rules, availability data, and deterministic financial calculations remain the foundation of the platform**.

This architecture also preserves a clean separation between the React frontend, Express backend, healthcare business services, AI/document-processing services, and PostgreSQL data layer, while keeping the implementation practical for the hackathon. The submitted project concept similarly positions React/Tailwind on the interface layer, Node/Express for APIs, Supabase/PostgreSQL for data, and Python/FastAPI plus OCR/LLM tooling for AI/document processing.

Document 2 complete: Full Technical Implementation Plan.

OpenHealth — Emergency Mode Architecture Update

Addendum to Document 2 — Full Technical Implementation Plan

This addendum preserves the existing 60-page implementation plan and adds the updated Emergency Mode architecture. It supersedes the earlier generic emergency-search flow wherever the two conflict.

Core behavior: Emergency Mode is an orchestration workflow, not a normal hospital-search form. The existing plan already includes Emergency.jsx, emergencyService.js, nearby discovery, availability, distance, capability filtering and ranking; this update extends those pieces into emergency sessions, ambulance matching and live tracking.

Safety rule: Never present an ambulance, hospital bed or hospital acceptance as guaranteed unless the corresponding provider/hospital confirmation has actually been received.

1. Updated Emergency Flow

```
USER CLICKS "EMERGENCY MODE"
↓
GET CURRENT LOCATION
↓
AUTOMATICALLY SEARCH NEARBY HOSPITALS
↓
FILTER FOR EMERGENCY CAPABILITY
↓
CHECK REPORTED ICU / EMERGENCY BED AVAILABILITY
↓
RANK SUITABLE HOSPITALS
↓
AUTOMATICALLY REQUEST / ARRANGE AMBULANCE
↓
MATCH AMBULANCE + HOSPITAL
↓
SHOW LIVE EMERGENCY STATUS
↓
USER CONFIRMS / TRACKS
```

Emergency.jsx should open immediately after the action and display the live orchestration state. It should not require the patient to first fill a standard hospital-search form.

2. Backend Emergency Services

```
backend/services/emergency/
■■■ emergencyService.js
■■■ hospitalMatchingService.js
■■■ ambulanceService.js
■■■ emergencyTrackingService.js
```

Service	Responsibility
emergencyService.js	Create/read/cancel sessions and orchestrate the end-to-end emergency workflow.
hospitalMatchingService.js	Find nearby hospitals; filter emergency capability; evaluate reported ICU/emergency availability; calculate distance and rank candid
ambulanceService.js	Create ambulance requests, integrate with provider adapters, assign vehicles and coordinate the destination hospital.
emergencyTrackingService.js	Aggregate session, hospital and ambulance states and expose normalized live status.

3. Emergency API Contract

Method	Endpoint	Purpose
POST	/api/emergency/session	Create emergency session and start orchestration.
GET	/api/emergency/session/:id	Return session and current orchestration state.
GET	/api/emergency/session/:id/hospitals	Return ranked emergency-capable hospital candidates and freshness metadata.
POST	/api/emergency/session/:id/ambulance	Request/arrange ambulance.
GET	/api/emergency/session/:id/ambulance	Return ambulance request/assignment/tracking state.
POST	/api/emergency/session/:id/cancel	Cancel session and cancellable external requests.
GET	/api/emergency/session/:id/status	Return normalized live emergency status.

The existing GET /api/emergency/search, POST /api/emergency/reserve and GET /api/emergency/nearby routes can remain for compatibility, but the session-based API is the primary Emergency Mode contract.

4. Emergency Database Extension

Add the following emergency entities to the database architecture. They integrate with the existing patient, hospital and bed entities rather than replacing them.

Table	Key fields / purpose
emergency_sessions	id; patient_id; requested_at; status; current_lat; current_lng; selected_hospital_id; selected_hospital_confirmed; ambulance_
emergency_hospital_matches	id; session_id; hospital_id; distance_km; emergency_capable; icu_available; emergency_bed_available; availability_last_upd
ambulance_providers	id; name; phone; service_areas; is_verified; created_at; updated_at.
ambulances	id; provider_id; vehicle_no; vehicle_type; equipment; is_available; current_lat; current_lng; created_at; updated_at.
ambulance_requests	id; session_id; provider_id; ambulance_id; driver_name; driver_phone; status; requested_at; assigned_at; eta_minutes; starte

5. Relationships

```
profiles
↓
patient_profiles
↓
emergency_sessions
■■■ emergency_hospital_matches ■■ hospitals ■■ hospital_beds ■■ bed_types
■■■ ambulance_requests ■■ ambulance_providers ■■ ambulances
■■■ destination / selected hospital
```

emergency_sessions is the correlation root. Every hospital match and ambulance request references the session so the backend can reconstruct the complete emergency journey and enforce ownership.

6. Hospital Matching

```
Current location
↓
Nearby hospital query
↓
Emergency capability filter
↓
Verification filter
↓
Reported ICU / emergency-bed availability
↓
Freshness validation
↓
Distance calculation
↓
Medical suitability
↓
Deterministic ranking
↓
Ranked candidates
```

The base plan already defines emergency discovery around availability, medical suitability, distance and verification. Emergency matching adds explicit session state and availability freshness so the patient can distinguish reported information from confirmed intake.

7. Emergency Status Lifecycle

Status	Meaning
Searching	Session created; nearby hospitals/resources are being evaluated.
Match Found	A suitable hospital candidate has been found; this is not yet hospital confirmation.
Request Sent	Ambulance request submitted to the configured provider/integration.
Ambulance Assigned	A specific ambulance has been confirmed by the provider.
Hospital Contacted	Hospital has been contacted/requested for emergency intake.
Hospital Confirmed	Hospital acceptance has been explicitly confirmed.
En Route	Ambulance is actively travelling.
Arrived	Ambulance has reached the relevant arrival point.
Cancelled	Session/request was cancelled.
Failed	A required orchestration step could not be completed.

Do not collapse these states. A hospital candidate is not the same as hospital confirmation, and an ambulance request is not the same as an ambulance assignment.

8. Ambulance Orchestration

```
Emergency Session
↓
Ambulance Required?
↓ yes
Create ambulance_request
↓
Provider / integration
↓
Request Sent
↓
Vehicle + driver matching
↓
Ambulance Assigned
↓
Destination hospital matching
↓
Hospital Contacted
↓
Hospital Confirmed
↓
En Route
↓
Arrived
```

For the hackathon, provider adapters may use seeded/demo data, but the same API contract and status lifecycle should be used. Production adapters can later call real ambulance providers.

9. Live Tracking

EmergencyTrackingService should aggregate the latest session, hospital-match and ambulance state. The frontend may use polling or realtime updates. Location, bed availability and provider state should carry timestamps whenever freshness matters.

10. Emergency RLS and Authorization

Actor	Emergency access
Patient	Read/create/update/cancel only their own emergency sessions and related patient-visible records.
Hospital staff	Access emergency requests/matches for their own hospital, subject to membership and minimum-necessary data.
Hospital admin	Manage emergency capability/operational data for their hospital and respond to hospital-contact workflows.
Insurance user	No general emergency-session access unless a specifically authorized workflow requires it.
Platform admin	Authorized operational/audit visibility according to platform policy.
Service role	Backend-only orchestration/integration access; never expose credentials to React.

RLS remains a database-layer control. Express role middleware is an additional application-layer control, not a replacement for RLS.

11. Emergency Constraints and Rules

Rule	Requirement
Ownership	patient_id must belong to the authenticated patient; clients cannot reassign ownership.
Location	Validate latitude/longitude before persistence.
Freshness	Availability used for ranking must include availability_last_updated_at.
Status	Reject invalid transitions; terminal states cannot silently become active again.
Auditability	Record creation, matching, selection, contact, confirmation, ambulance assignment, cancellation and failure.
No guarantee	Candidate match/request sent must never be rendered as confirmed.
Integrity	Use foreign keys between emergency tables and existing patient/hospital/bed entities.

12. Emergency Audit Events

```
EMERGENCY_SESSION_CREATED
EMERGENCY_LOCATION_CAPTURED
EMERGENCY_HOSPITAL_SEARCHED
EMERGENCY_HOSPITAL_MATCHED
EMERGENCY_HOSPITAL_SELECTED
EMERGENCY_HOSPITAL_CONTACTED
EMERGENCY_HOSPITAL_CONFIRMED
AMBULANCE_REQUEST_CREATED
AMBULANCE_ASSIGNED
AMBULANCE_STATUS_UPDATED
EMERGENCY_DESTINATION_CHANGED
EMERGENCY_CANCELLED
EMERGENCY_FAILED
```

13. Frontend Emergency Components

```
frontend/src/components/emergency/
■■■ EmergencyActionCard.jsx
■■■ EmergencyStatus.jsx
■■■ NearbyHospital.jsx
■■■ AmbulanceStatus.jsx
■■■ EmergencyMap.jsx

frontend/src/pages/patient/
■■■ Emergency.jsx
```

Component	Role
EmergencyActionCard.jsx	Entry point and session initiation/confirmation.
EmergencyStatus.jsx	Primary status timeline, warnings and confirmation state.
NearbyHospital.jsx	Ranked hospital candidates, emergency capability, reported bed state and freshness.
AmbulanceStatus.jsx	Request/assignment/ETA/tracking state.
EmergencyMap.jsx	Patient, hospital and ambulance map/route when available.
Emergency.jsx	Dedicated orchestration page; mobile-first emergency experience.

14. Frontend State Machine

```
INITIAL
↓
CREATING SESSION
↓
SEARCHING HOSPITALS
↓
MATCH FOUND
↓
REQUEST SENT
↓
AMBULANCE ASSIGNED
↓
HOSPITAL CONTACTED
↓
HOSPITAL CONFIRMED
↓
EN ROUTE
↓
ARRIVED

Any active state → CANCELLED / FAILED
```

The status timeline is the primary operational surface. The map is supporting context, not a substitute for confirmed state.

15. Emergency Response Shape

```
{
success: true,
data: {
session: {
id,
status,
currentLocation: { lat, lng },
ambulanceRequired
},
hospitals: [
{
hospitalId,
name,
distanceKm,
emergencyCapable,
icuAvailable,
emergencyBedAvailable,
availabilityLastUpdatedAt,
matchScore,
rank,
status
}
],
```

```
ambulance: {
status,
providerId,
ambulanceId,
etaMinutes,
lastUpdatedAt
}
}
}
```

16. Emergency Migration Extension

```
039_emergency_enums.sql
040_emergency_sessions.sql
041_emergency_hospital_matches.sql
042_ambulance_providers.sql
043_ambulances.sql
044_ambulance_requests.sql
045_emergency_rls.sql
046_emergency_indexes.sql
047_emergency_triggers.sql
048_emergency_seed.sql
```

The existing migration sequence through 038_seed.sql remains valid; these migrations extend the baseline.

17. Emergency Indexes

```
emergency_sessions(patient_id, status)
emergency_sessions(created_at)
emergency_hospital_matches(session_id, rank_position)
emergency_hospital_matches(hospital_id, status)
emergency_hospital_matches(availability_last_updated_at)
ambulance_requests(session_id, status)
ambulance_requests(provider_id, status)
ambulances(provider_id, is_available)
```

18. Emergency Triggers

Trigger / function	Purpose
updated_at trigger	Maintain updated_at on emergency records.
status-transition validation	Reject invalid state changes.
audit event writer	Record critical emergency state changes.
availability freshness validation	Prevent stale/missing data from being treated as current.
ownership protection	Prevent unauthorized patient/session ownership reassignment.
referential integrity	Enforce valid emergency-to-patient/hospital/provider/vehicle relationships.

Complex orchestration remains in backend services; database triggers should enforce invariants, timestamps, ownership protection and audit requirements.

19. Updated Emergency API Request Flow

```
Emergency.jsx
↓
emergencyService.js
↓
POST /api/emergency/session
↓
emergencyController
↓
emergencyService
■■■ hospitalMatchingService
■ ■■■ nearby hospitals
■ ■■■ emergency capability
■ ■■■ bed availability
■ ■■■ freshness
■ ■■■ ranking
■■■ ambulanceService
■ ■■■ provider request
■ ■■■ ambulance matching
■ ■■■ destination coordination
■■■ emergencyTrackingService
■■■ normalized live status
↓
PostgreSQL + RLS
↓
EmergencyStatus / NearbyHospital / AmbulanceStatus / EmergencyMap
```

20. Error and Uncertainty Handling

Condition	Required behavior
Location permission denied	Explain the requirement and use a manual fallback if supported.
No emergency-capable hospital	Show a clear failure state and alternatives without claiming emergency acceptance.
Stale bed data	Display stale/uncertain availability with its timestamp.
Hospital not confirmed	Display Hospital Contacted, not Hospital Confirmed.
Ambulance pending	Display Request Sent and continue tracking.
No ambulance available	Show transport unavailable/failed state and preserve hospital alternatives.
Provider timeout	Do not claim assignment; retry or fail according to backend policy.
Cancelled	Stop active tracking and show cancellation state.

21. Mobile-First Emergency UX

Emergency Mode is particularly optimized for mobile access. Prioritize current location, selected/ranked hospital, ambulance state and one clear action. Avoid dense forms at entry. Use large touch targets and concise status labels.

22. Emergency Integration with Existing Architecture

Existing area	Emergency reuse
hospitals	Identity, coordinates, verification and emergency capability.
departments	Emergency department capability through emergency_available.
hospital_beds	ICU/general/emergency resource counts and freshness.
bed_reservations	Existing reservation logic remains separate from emergency intake confirmation.
profiles / patient_profiles	Emergency session ownership and patient identity.
notifications	Emergency status notifications where appropriate.
audit_logs	Trace critical emergency actions.
Maps	Nearby discovery, distance and route information.
Supabase PostgreSQL / RLS	Persistent state, constraints, relationships and authorization.

23. Updated Hackathon Development Priority

Emergency Mode remains a Priority 1 capability in the base plan. Implement the emergency session, hospital matching, reported bed availability, ambulance request/assignment and live status as one coherent flow rather than separate CRUD screens.

```
Recommended demo:
Patient clicks Emergency Mode
→ location captured
→ nearby emergency-capable hospitals discovered
→ ICU/emergency bed data evaluated with freshness
→ hospital ranked
→ ambulance request sent
→ ambulance assigned (demo/provider response)
→ hospital contacted
→ hospital confirmation shown only after confirmation
→ en-route status and map displayed
→ arrival/cancellation/failure handled explicitly.
```

24. Final Emergency Architecture

```
React Emergency.jsx
↓
emergencyService.js
↓
Emergency Controller
↓
Hospital Matching ■■■■■ Ambulance Service
↓ ↓
Hospitals + Beds Provider + Vehicles
■■■■■■■■■■■■■■■■■■■■■■■■■■
↓
Emergency Tracking Service
↓
PostgreSQL + RLS
↓
Live Emergency UI
```

This extension fits the original technical plan: React owns presentation, Express services own business orchestration, Supabase PostgreSQL remains persistent state, Maps/ambulance systems remain external adapters, and RLS/constraints protect sensitive records.